

Universidad San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Curso: Análisis y Diseño de Sistemas 1
Sección: A
Ingeniero: Daniel Alberto Gonzáles Gonzáles



**MANUAL TÉCNICO
PRÁCTICA 1 - GRUPO 3
SGG**

Nombre: Angel Abelino Son Vicente.....**Carnet:** 202030885
Nombre: Dany Noé de León Santizo.....**Carnet:** 201830006
Nombre: Mario Raúl Yancor Ocaña.....**Carnet:** 201930761
Nombre: Herberth Julian Reyes Pacajoj.....**Carnet:** 202230236

Quetzaltenango, agosto 2026

ÍNDICE

INTRODUCCIÓN.....	2
OBJETIVOS.....	3
DESCRIPCIÓN GENERAL DEL SISTEMA.....	4
TECNOLOGÍAS USADAS.....	5
ARQUITECTURA DEL SISTEMA.....	6
Arquitectura.....	6
ESTRUCTURA DEL PROYECTO.....	7
COMPONENTES.....	10
BASE DE DATOS.....	11
Modelo de datos.....	12
DIAGRAMA DE CLASES.....	19
CASOS DE USO.....	20
DIAGRAMA DE SECUENCIA.....	21
REPORTE DE ASISTENCIA Y EXPORTACIONES.....	22
Reporte de asistencia por clase.....	22
SEGURIDAD.....	23
Control de acceso por roles.....	23
Autenticación de dos factores.....	23
INSTALACIÓN.....	24
Requisitos.....	24
Clonar proyecto.....	24
Dependencias PHP.....	24
Configuración del entorno.....	24
Base de datos.....	24
Dependencias frontend.....	25
Construcción.....	25
Ejecutar servidor.....	25
PRUEBAS.....	26
CONCLUSIONES.....	27

INTRODUCCIÓN

El presente Manual Técnico documenta la estructura, configuración y funcionamiento interno del Sistema de Gestión de Gimnasio (SGG), desarrollado como parte del curso de Análisis y Diseño de Sistemas 1.

El sistema fue desarrollado como una aplicación web que permite centralizar diferentes procesos administrativos y operativos de un gimnasio. Entre sus principales funciones se encuentran la administración de usuarios y roles, gestión de socios y empleados, membresías, pagos, asistencia, clases grupales, asignación de entrenadores, seguimiento físico, nutrición, pases de invitado y generación de reportes.

La aplicación está desarrollada utilizando el framework Laravel bajo una arquitectura organizada en capas, utilizando controladores para recibir las solicitudes del usuario, servicios para la lógica de negocio, modelos Eloquent para el acceso a los datos y vistas Blade para la presentación de la información.

Este documento está dirigido principalmente a desarrolladores, administradores técnicos y personas encargadas del mantenimiento del sistema. Su propósito es facilitar la instalación del proyecto, comprensión de su arquitectura, estructura de base de datos, principales módulos, configuraciones, pruebas y procedimientos necesarios para continuar su desarrollo o realizar mantenimiento.

OBJETIVOS

Objetivos general

Documentar de forma técnica la estructura, configuración, funcionamiento e implementación del Sistema de Gestión de Gimnasio, proporcionando la información necesaria para su instalación, mantenimiento y futura ampliación.

Objetivos específicos

- Describir las tecnologías y herramientas utilizadas para el desarrollo del sistema.
- Explicar la arquitectura y organización interna del proyecto.
- Documentar la estructura y relaciones de la base de datos.
- Describir los principales módulos, controladores, servicios y modelos.
- Documentar los mecanismos de autenticación, autorización y seguridad.

DESCRIPCIÓN GENERAL DEL SISTEMA

Descripción general

El Sistema de Gestión de Gimnasio es una aplicación web diseñada para apoyar la administración de las operaciones principales de un gimnasio.

El sistema utiliza diferentes roles para restringir las funcionalidades disponibles para cada usuario:

Administrador: posee acceso a las funciones administrativas del sistema, tales como gestión de empleados, socios, clases grupales, asignaciones de entrenadores, catálogos y reportes.

Recepcionista: realiza operaciones relacionadas con la atención de los socios, entre ellas inscripciones permitidas, pases de invitado, asistencia y otros procesos autorizados.

Entrenador: administra las actividades relacionadas con sus socios asignados y las clases grupales bajo su responsabilidad, incluyendo rutinas, mediciones, observaciones y registro de asistencia.

Socio: accede a las funcionalidades destinadas al usuario final, tales como consulta e inscripción a clases grupales, registro de comidas, seguimiento de entrenamiento y otras funciones asociadas a su membresía.

El sistema utiliza autenticación y control de acceso basado en roles. Además, el inicio de sesión incluye un proceso de doble factor de autenticación para aumentar la seguridad del acceso.

TECNOLOGÍAS USADAS

Tecnología	Uso dentro del proyecto
PHP 8.3 o superior	Lenguaje principal del backend
Laravel 13	Framework principal
Blade	Motor de plantillas
Bootstrap 5.3	Diseño y componentes visuales
Bootstrap Icons	Iconografía
JavaScript	Funcionalidades del cliente
Vite 8	Gestión y compilación de recursos
SQLite	Base de datos utilizada en desarrollo
Eloquent ORM	Acceso y relaciones de base de datos
Composer	Gestión de dependencias PHP
NPM	Gestión de dependencias frontend
PHPUnit	Pruebas automatizadas
Laravel Breeze	Base para autenticación
Laravel DOMPDF	Generación de reportes PDF
Laravel Excel	Exportación XLSX/CSV
html2canvas	Exportación visual de reportes en PNG
Git/GitHub	Control de versiones

```

PS C:\Users\mario\Desktop\AyD1-PrimeraPractica\AyD1-SGG> php -v
PHP 8.4.24 (cli) (built: Jul 29 2026 06:00:34) (NTS Visual C++ 2022 x64)
Copyright (c) The PHP Group
Built by The PHP Group
Zend Engine v4.4.24, Copyright (c) Zend Technologies
PS C:\Users\mario\Desktop\AyD1-PrimeraPractica\AyD1-SGG> composer --version
Composer version 2.9.7 2026-04-14 13:31:52
PHP version 8.4.24 (C:\php84\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.
PS C:\Users\mario\Desktop\AyD1-PrimeraPractica\AyD1-SGG> node -v
v22.12.0
PS C:\Users\mario\Desktop\AyD1-PrimeraPractica\AyD1-SGG> npm -v
10.9.0
PS C:\Users\mario\Desktop\AyD1-PrimeraPractica\AyD1-SGG> php artisan --version
Laravel Framework 13.25.0

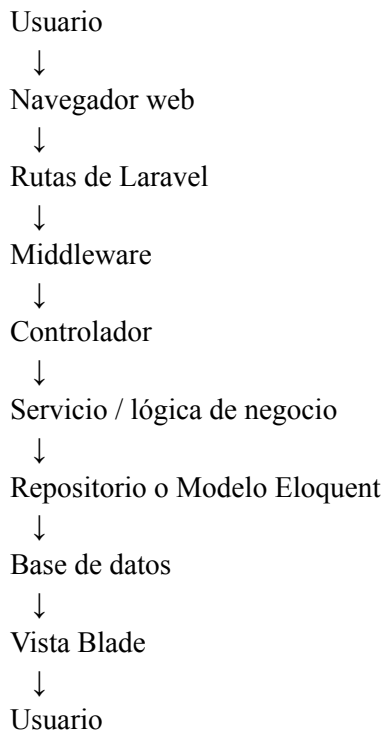
```

ARQUITECTURA DEL SISTEMA

Arquitectura

El sistema sigue una arquitectura web organizada en diferentes capas con el objetivo de separar responsabilidades y facilitar el mantenimiento del código.

El flujo general de una solicitud se puede representar de la siguiente manera:



Las capas principales son:

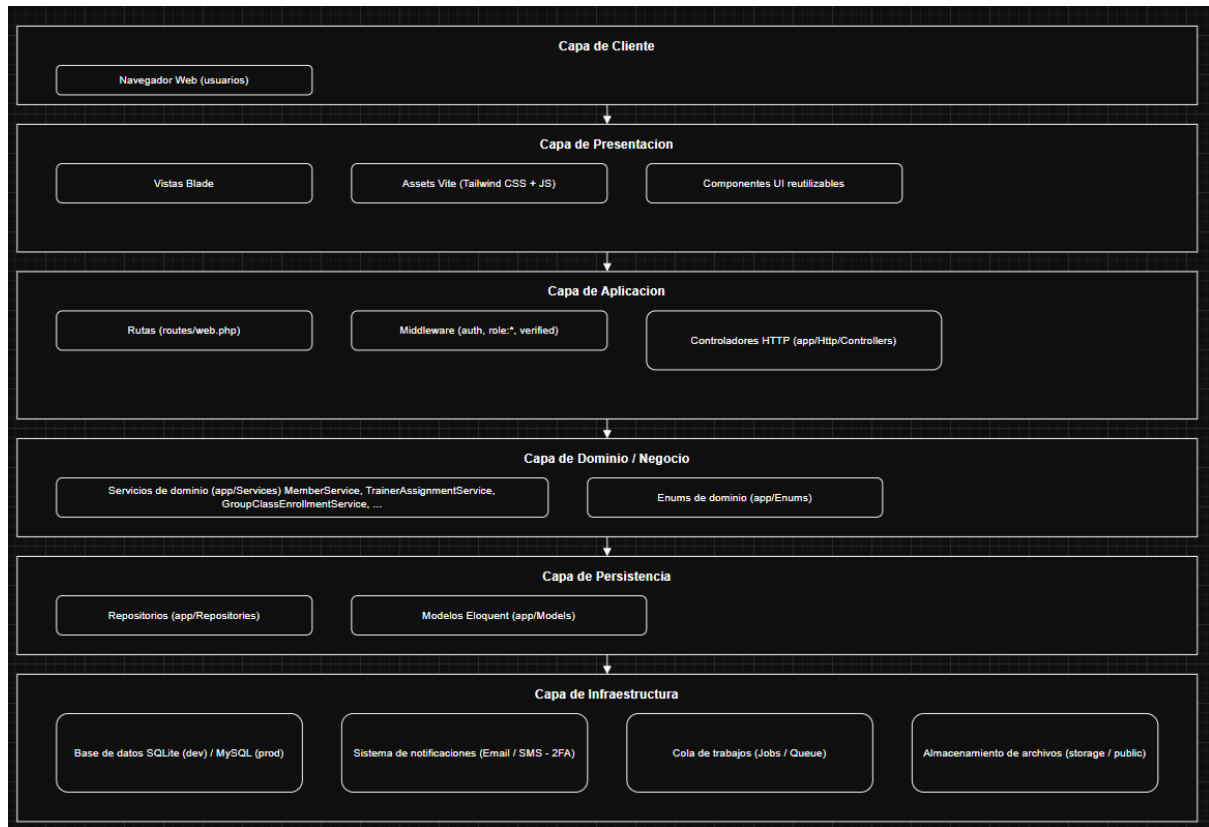
Capa de presentación. Está formada principalmente por las vistas Blade ubicadas en `resources/views`. La interfaz utiliza Bootstrap para los elementos visuales y Vite para administrar los recursos de la aplicación.

Capa de aplicación. Incluye las rutas, middleware y controladores. Es la encargada de recibir las solicitudes HTTP, comprobar permisos y coordinar las acciones del sistema.

Capa de negocio. Está compuesta principalmente por los servicios ubicados en `app/Services`. En ellos se implementan reglas como validación de membresías, control de cupos, límites de clases, pagos, nutrición y asignaciones.

Capa de persistencia. Está formada por los modelos Eloquent y repositorios utilizados para consultar y modificar la información almacenada.

Capa de infraestructura. Incluye la base de datos, sistema de notificaciones, almacenamiento, colas y demás componentes utilizados para dar soporte a la aplicación.



ESTRUCTURA DEL PROYECTO

AyD1-SGG/

|

|— app/

| |— Console/

| |— Enums/

| |— Exports/

| |— Http/

| | |— Controllers/

| | |— Middleware/

| | |— Requests/

| |— Models/


```
| |— Notifications/
| |— Repositories/
| |— Services/
| |— Providers/
|
|— bootstrap/
|— config/
|— database/
| |— factories/
| |— migrations/
| |— seeders/
|
|— public/
|— resources/
| |— css/
| |— js/
| |— views/
|
|— routes/
|— storage/
|— tests/
|— composer.json
|— package.json
|— artisan
```

app/Http/Controllers: contiene los controladores responsables de recibir las solicitudes del usuario.

app/Models: contiene los modelos Eloquent que representan las entidades almacenadas en la base de datos.

app/Services: concentra lógica de negocio que no debe permanecer directamente en los controladores.

app/Repositories: contiene clases utilizadas para encapsular determinadas operaciones de acceso a datos.

app/Enums: almacena enumeraciones utilizadas para representar estados y tipos válidos.

app/Exports: contiene las clases encargadas de preparar información para exportaciones Excel.

app/Notifications: contiene las notificaciones enviadas por el sistema.

database/migrations: define la estructura y modificaciones realizadas sobre las tablas.

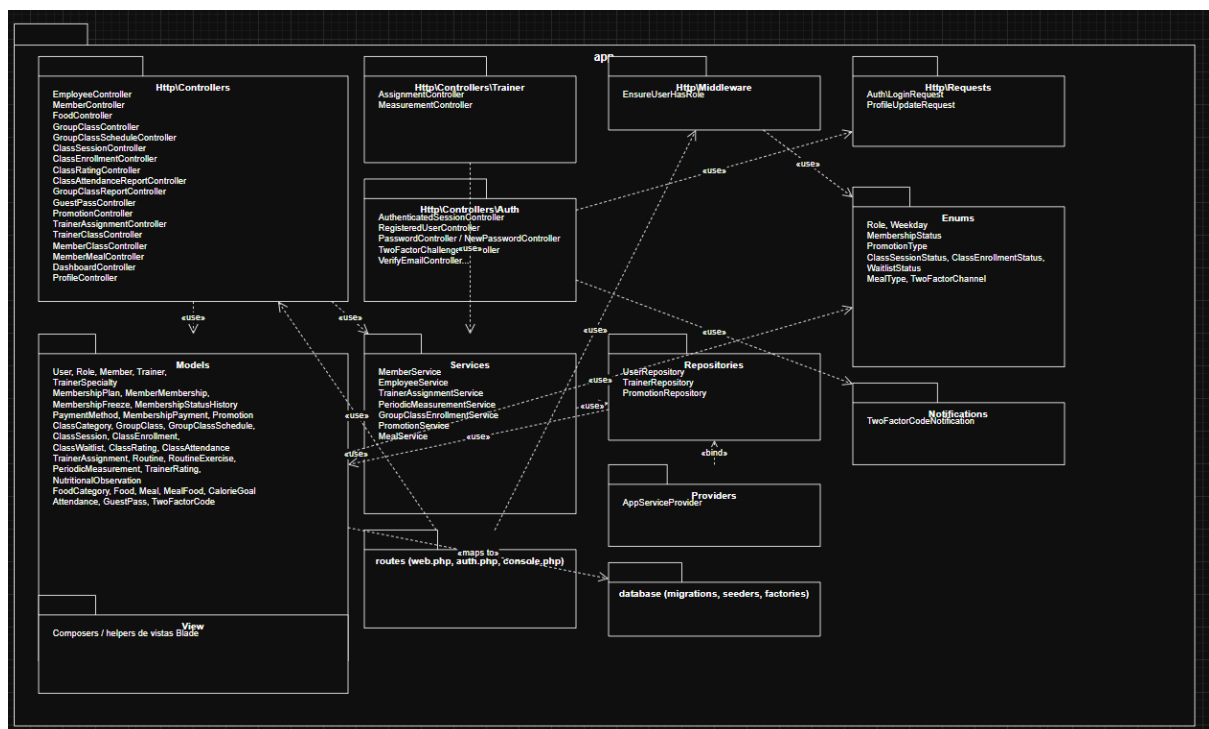
database/seeders: contiene datos utilizados para inicializar el sistema.

resources/views: contiene las interfaces construidas mediante Blade.

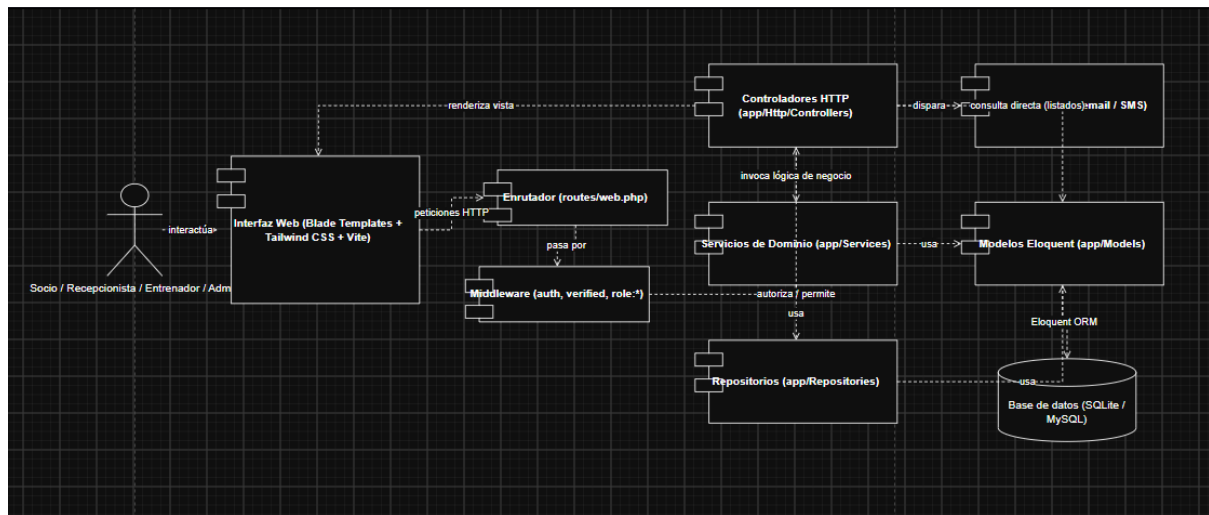
routes/web.php: contiene las rutas principales de la aplicación.

routes/auth.php: contiene las rutas de autenticación, recuperación de contraseña, verificación de correo y 2FA.

tests/Feature: contiene las pruebas funcionales realizadas sobre los diferentes módulos.



COMPONENTES



Interfaz web



Enrutador



Middleware



Controladores



Servicios



Repositorios / Modelos



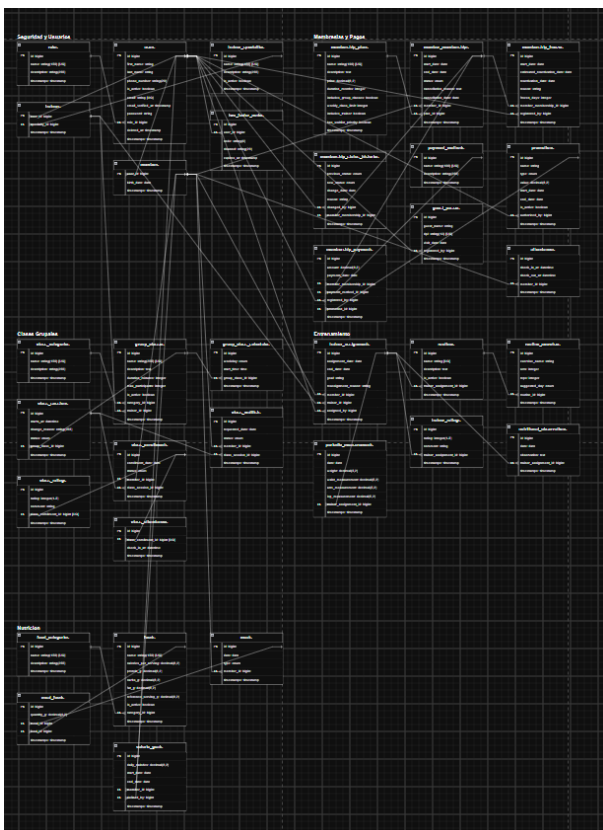
Base de datos

BASE DE DATOS

Diagrama E/R



Diagrama Tablas



Modelo de datos

0. Usuarios y roles

```
roles (Rol) (
  id
  name (nombre) → string, unique
  description (descripcion) → string, nullable
)
```

```
users (Usuario) (
  id,
  first_name (nombre) → string
  last_name (apellido) → string
  email → string, unique
  password → string
  phone_number (telefono) → string[20], nullable
  is_active (activo) → boolean
  role_id → llave foránea a roles
)
```

- **users** implementa SoftDeletes (método de Eloquent)

```
trainer_specialties (Especialidad_Entrenador) (
  id
  name (nombre) → string, unique
  description (descripcion) → string, nullable
  is_active (activo) → boolean
)
```

```
trainers (Entrenador) (
  user_id → llave foránea a users → usa el mismo id (herencia/especialización)
  specialty_id → llave foránea a trainer_specialties
)
```

```
members (Miembro) (
  user_id → llave foránea a users → usa el mismo id (herencia/especialización)
  birth_date (fecha_nacimiento) → date
)
```

1. Membresías

```
membership_plans (Plan_Membresia) (
  id,
  name (nombre) → string, unique
  description (descripcion) → string, nullable
  price (precio) → decimal(8,2)
  duration_months (duracion_meses) → integer
  includes_group_classes (incluye_clases_grupales) → boolean
  weekly_class_limit (limite_clases) → integer, nullable
)
```

includes_trainer (incluye_entrenador) → boolean
 has_waitlist_priority (tiene_prioridad_lista_espera) → boolean

)

- duration_months → mensual (1), trimestral (3), anual (12)
- includes_group_classes → true para Premium y Élite
- weekly_class_limit → null = ilimitado, diferenciado de Básico por includes_group_classes = true
- includes_trainer → true para Élite
- has_waitlist_priority → true para Élite

member_memberships (Membresía) (

id,
 start_date (fecha_inicio) → date
 end_date (fecha_fin) → date
 cancellation_reason (razon_cancelacion) → text, nullable
 cancellation_date (fecha_cancelacion) → date, nullable
 expiration_warning_sent
 expiration_warning_sent
 status (estado) → enum('active', 'frozen', 'expired', 'cancelled')
member_id → llave foránea a **members**
plan_id → llave foránea a **membership_plans**

)

- status → estado de la membresía: Activa, Congelada, Vencida, Cancelada

membership_freezes (Congelamiento_Membresía) (

id,
 start_date (fecha_inicio) → date
 estimated_reactivation_date (fecha_estimada_reactivacion) → date, nullable
 reactivation_date (fecha_reactivacion) → date, nullable
 reason (motivo) → string
 frozen_days (dias_congelados) → integer
member_membership_id → llave foránea a **member_memberships**
registered_by → llave foránea a **users** (quién registró la acción)

)

membership_status_histories (Historial_Estado_Membresía) (

id,
 previous_status (estado_previo) → enum('active', 'frozen', 'expired', 'cancelled'), nullable
 new_status (estado_nuevo) → enum('active', 'frozen', 'expired', 'cancelled')
 reactivation_date (fecha_reactivacion) → date, nullable
 change_date (fecha_cambio) → date
 reason (motivo) → string, nullable
changed_by → llave foránea a **users** (quién realizó el cambio)
member_membership_id → llave foránea a **member_memberships** (qué membresía sufrió el

cambio)

)

```
payment_methods (Metodo_Pago) (
    id
    name (nombre) → string, unique
    description (descripcion) → string, nullable
)
```

```
promotions (Descuento_Promocion) (
    id,
    name (nombre) → string
    type (tipo) → enum('percentage', 'fixed_amount')
    value (valor) → decimal(8,2)
    start_date (fecha_inicio) → date
    end_date (fecha_fin) → date
    is_active (activa) → boolean
    authorized_by → llave foránea a users (qué admin creó la promoción)
)
```

- type → tipo de promoción: Porcentaje, Monto fijo
- value → valor de descuento (ej. Si type= 'percentage' y value=5 → descuento del 5%)

```
membership_payments (Pago_Membresia) (
    id,
    amount (monto) → decimal(8,2)
    payment_date (fecha_pago) → date
    member_membership_id → llave foránea a member_memberships (la membresía)
    payment_method_id → llave foránea a payment_methods (el método de pago)
    registered_by → llave foránea a users (quién registró la acción)
    promotion_id → llave foránea a promotions (null si no se aplicó ninguna promoción)
)
```

2. Clases grupales

```
class_categories (Categoria_Clase) (
    id
    name (nombre) → string, unique
    description (descripcion) → string, nullable
)
```

```
group_classes (Clase_Gupal) (
    id,
    name (nombre) → string, unique
    description (descripcion) → string, nullable
    duration_minutes (duracion_minutos) → integer
    max_participants (maximo_participantes) → integer
    is_active (activo) → boolean
    category_id → llave foránea a class_categories (qué categoría: Cardio, Fuerza, etc)
    trainer_id → llave foránea a trainers (qué entrenador va a impartir la clase)
)
```

)

```
group_class_schedules (Dias_Clase_Grupal) (
    id,
    weekday (dia) → enum('monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday',
'sunday')
    start_time (hora_inicio) → time
    group_class_id → llave foránea a group_classes (clase a la que se le programa una sesion)
)
```

```
class_sessions (Sesion_Clase) (
    id,
    starts_at (empieza) → dateTime
    change_reason (razon_cambio) → string, nullable
    status (estado) → enum ('scheduled', 'in_progress', 'completed', 'cancelled', 'rescheduled')
    group_class_id → llave foránea a group_classes (clase a la que pertenece la sesion)
)
```

- starts_at → puede tomar la hora de start_time (**group_class_schedules**) si comenzó a la hora agenda, sino (se reprogramo) toma otra hora
- status → estado de la sesión: Programada, En progreso, Completada, Cancelada, Reprogramada

```
class_enrollments (Inscripcion_Clase) (
    id,
    enrollment_date (fecha_inscripcion) → date
    status (estado) → enum ('enrolled', 'cancelled', 'attended', 'no_show')
    member_id → llave foránea a members (miembro que se inscribió)
    class_session_id → llave foránea a class_sessions (sesión de clase a la que se inscribió)
)
```

- status → estado de la inscripción: Inscrito, Cancelado, Asistió, No asistió

```
class_waitlists (Lista_Espera) (
    id,
    requested_date (fecha_solicitud) → date
    status (estado) → enum ('waiting', 'notified', 'expired')
    member_id → llave foránea a members (miembro que esta en la lista de espera)
    class_session_id → llave foránea a class_sessions (sesión de clase a la que se está esperando
por un cupo disponible)
)
```

- status → estado del miembro en espera: Esperando, Notificado, Expirado

```
class_ratings (Calificacion_Clase) (
    id,
    rating (calificacion) → integer (1-5)
```


comment (comentario) → string, nullable
class_enrollment_id → llave foránea a **class_enrollments** (sirve para comprobar que alguien que se inscribió a la clase es quién da la calificación)
)

3. Registro de asistencia

attendances (Asistencia) (
id,
 check_in_at (hora_entrada) → dateTime
 check_out_at (hora_salida) → dateTime, nullable
member_id → llave foránea a **members** (miembro que asiste)
)

guest_passes (Pase_Invitado) (
id,
 guest_name (nombre_invitado) → string
 dpi → string[13], unique
 visit_date (fecha_visita) → date
registered_by → llave foránea a **users** (quién registró la acción)
)

4. Asignación de entrenador

trainer_assignments (Asignacion_Entrenador) (
id,
 assignment_date (fecha_asignacion) → date
 end_date (fecha_fin) → date, nullable
 goal (objetivo) → string, nullable
 reassignment_reason (razon_reasignacion) → string, nullable
member_id → llave foránea a **members** (miembro asignado)
trainer_id → llave foránea a **trainers** (entrenador asignado)
assigned_by → llave foránea a **users** (quién realizó la asignación)
)

routines (Rutina) (
id,
 name (nombre) → string, unique
 description (descripcion) → string, nullable
 is_active (activa) → boolean
trainer_assignment_id → llave foránea a **trainer_assignments** (para saber qué entrenador asignó la rutina a qué miembro)
)

routine_exercises (Ejercicio_Rutina) (
id,
 exercise_name (nombre_ejercicio) → string

```

    sets → integer
    reps (repeticiones) → integer
    suggested_day (dia_sugerido) → enum('monday', 'tuesday', 'wednesday', 'thursday', 'friday',
'saturday', 'sunday')
    routine_id → llave foránea a routines (rutina a la que pertenece el ejercicio)
)

```

```

periodic_measurements (Medicion_Periodica) (
    id,
    date (fecha) → date
    weight (peso) → decimal(5,2)
    waist_measurement (medida_cintura) → decimal(5,2), nullable
    arm_measurement (medida_brazo) → decimal(5,2), nullable
    leg_measurement (medida_pierna) → decimal(5,2), nullable
    trainer_assignment_id → llave foránea a trainer_assignments (para saber qué entrenador
registró las medidas de qué miembro)
)

```

```

trainer_ratings (Calificacion_Entrenador) (
    id,
    rating (calificacion) → integer (1-5)
    comment (comentario) → string, nullable
    trainer_assignment_id → llave foránea a trainer_assignments (sirve para comprobar que
alguien que estaba asignado al entrenador es quién da la calificación)
)

```

5. Nutrición

```

food_categories (Categoria_Alimentos) (
    id
    name (nombre) → string, unique
    description (descripcion) → string, nullable
)

```

```

foods (Catalogo_Alimentos) (
    id
    name (nombre) → string, unique
    calories_per_serving (calorias_por_porcion) → decimal(8,2)
    protein_g (proteina_g) → decimal(8,2)
    carbs_g (carbohidratos_g) → decimal(8,2)
    fat_g (grasas_g) → decimal(8,2)
    reference_serving_g (porcion_referencia_g) → decimal(8,2)
    is_active (activo) → boolean
    category_id → llave foránea a food_categories (qué categoría: Frutas, Verduras, etc)
)

```

```

meals (Comida_Registrada) (
    id

```

date (fecha) → date
 type (tipo) → enum('breakfast', 'lunch', 'dinner', 'snack')
member_id → llave foránea a **members** (miembro que registra lo que consumió)

)

- type → tipo de comida: Desayuno, Almuerzo, Cena, Snack

meal_foods (Comida_Alimento) (
id,
 quantity_g (cantidad_g) → decimal(6,2)
meal_id → llave foránea a **meals** (comida que registró (ej. desayuno))
food_id → llave foránea a **foods** (los alimentos que consumió en la comida (ej. huevo, café))

)

- Esta tabla funciona como pivote (N:M), una comida tiene muchos alimentos y un alimento está en varias comidas
- UNIQUE(meal_id, food_id): evita que el mismo alimento se registre dos veces en la misma comida

calorie_goals (Meta_Calorica) (
id,
 daily_calories (meta_calorica) → decimal(8,2)
 start_date (fecha_inicio) → date
 end_date (fecha_fin) → date, nullable
member_id → llave foránea a **members** (miembro con la meta calórica)
defined_by → llave foránea a **users** (quién definió la meta: miembro o entrenador)

)

nutritional_observations (Observacion_Nutricional) (
id,
 date (fecha) → date
 observation (observacion) → text
trainer_assignment_id → llave foránea a **trainer_assignments** (sirve para comprobar que un entrenador hizo la observación a un miembro que él tiene asignado)

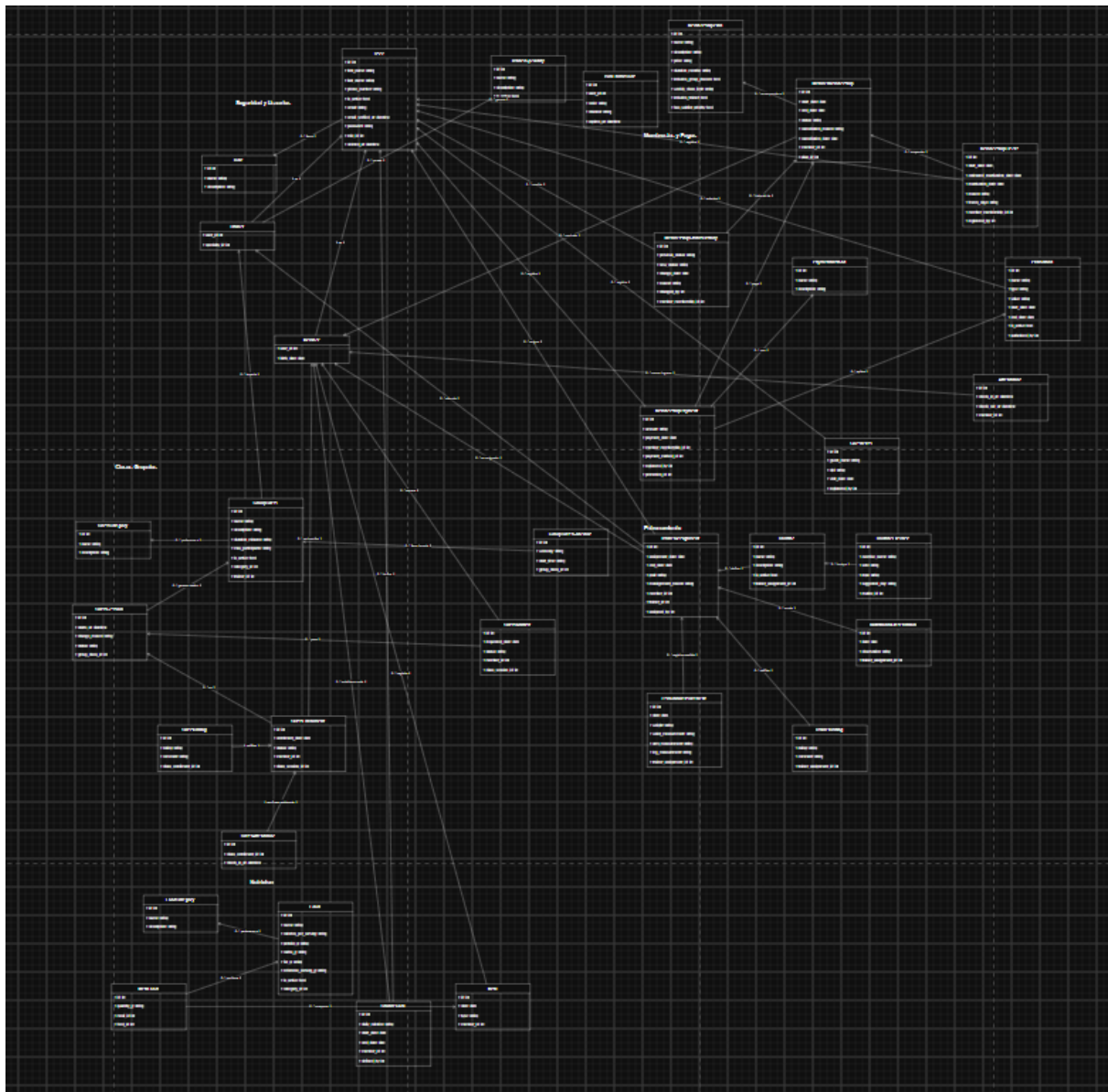
)

Enums de PHP (app/Enums/)

Estos enums mapean 1:1 con las columnas `enum` de MySQL correspondientes, y se aplican como cast en el modelo Eloquent.

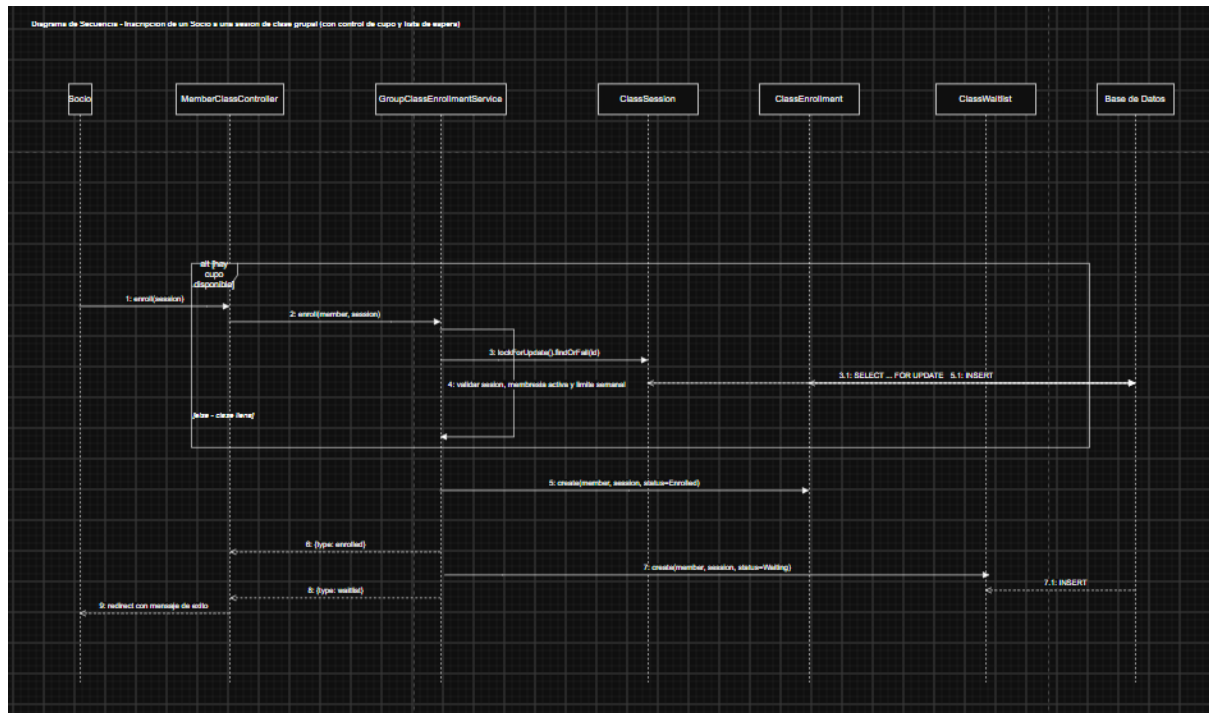
- MembershipStatus → usado en member_memberships.status, membership_status_histories.previous_status/new_status
- ClassSessionStatus → usado en class_sessions.status
- ClassEnrollmentStatus → usado en class_enrollments.status
- WaitlistStatus → usado en class_waitlists.status
- PromotionType → usado en promotions.type
- Weekday → usado en group_class_schedules.weekday, routine_exercises.suggested_day

DIAGRAMA DE CLASES



El diagrama de clases representa las principales entidades implementadas mediante modelos Eloquent, sus atributos y las relaciones utilizadas para acceder a la información del sistema. Estas relaciones permiten representar asociaciones como usuarios y roles, socios y membresías, entrenadores y asignaciones, así como clases, sesiones, inscripciones y asistencia.

DIAGRAMA DE SECUENCIA



Inscripción de un socio a una sesión de clase grupal con control de cupo y lista de espera.

Tu diagrama muestra:

Socio



MemberClassController



GroupClassEnrollmentService



ClassSession



ClassEnrollment / ClassWaitlist



BD

REPORTE DE ASISTENCIA Y EXPORTACIONES

Reporte de asistencia por clase

El administrador dispone de un reporte de asistencia de clases grupales que permite aplicar filtros por:

Fecha inicial

Fecha final

Clase grupal

El reporte muestra:

Clase

Fecha

Hora

Socio

Correo

Check-in

Estado

y calcula:

Cantidad de participantes

Asistieron

No asistieron

Porcentaje de asistencia

El reporte permite exportación en:

PDF

Excel (.xlsx)

Imagen (.png)

La generación de Excel utiliza:

ClassAttendanceReportExport

Maatwebsite Excel

La generación PDF utiliza:

Laravel DOMPDF

class-attendance-reports/pdf.blade.php

y la imagen se obtiene desde el navegador mediante:

html2canvas

SEGURIDAD

Control de acceso por roles

El proyecto tiene el middleware:

`app/Http/Middleware/EnsureUserHasRole.php`

que verifica el rol antes de permitir el acceso.

Ejemplo:

```
Route::middleware(['auth', 'role:admin'])->group(function () {  
    // rutas administrativas  
});
```

Cuando el usuario no posee un rol autorizado, el sistema responde con:

403 Forbidden

Autenticación de dos factores

Las rutas de autenticación incluyen:

`two-factor-challenge`
`two-factor-challenge/send`

por medio de:

`TwoFactorChallengeController`
`TwoFactorCode`

De esta manera, las credenciales por sí solas no completan el ingreso al sistema.

INSTALACIÓN

Requisitos

PHP >= 8.3

Composer

Node.js

NPM

SQLite

Git

Y para las exportaciones Excel debe estar habilitada:

`extension=gd`

en `php.ini`, porque PhpSpreadsheet la requiere.

Clonar proyecto

```
git clone <URL_DEL_REPOSITORIO>
```

```
cd AyD1-SGG
```

Dependencias PHP

```
composer install
```

Configuración del entorno

En Windows:

```
Copy-Item .env.example .env
```

o:

```
cp .env.example .env
```

Generar llave:

```
php artisan key:generate
```

Base de datos

El proyecto está preparado por defecto para:

```
DB_CONNECTION=sqlite
```

Crear el archivo si no existe:

```
New-Item database\database.sqlite
```

Migrar:

php artisan migrate

Si se desean datos iniciales:

php artisan db:seed

Dependencias frontend

npm install

Construcción

npm run build

O para desarrollo:

npm run dev

Ejecutar servidor

php artisan serve

Dirección normalmente:

<http://127.0.0.1:8000>

PRUEBAS

```
PS C:\Users\mario\Desktop\AyD1-PrimeraPractica\AyD1-SGG> php artisan test

PASS Tests\Feature\MembershipFreezeTest
✓ quota calculation ignores older freezes
✓ quota calculation includes overlapping portions
✓ quota calculation includes open freezes
✓ cannot freeze if no remaining quota
✓ cannot freeze longer than remaining quota
✓ defaults to 15 days and fails if quota is insufficient
✓ cancellation closes active freeze correctly

PASS Tests\Feature\NutritionalObservationTest
✓ trainer with active assignment can add observation
✓ trainer cannot add observation for an assignment that is not theirs
✓ trainer without active assignment cannot view members page
✓ todays meal log is visible to the assigned trainer
✓ member sees trainer observations on nutrition history
✓ member without trainer sees empty state

PASS Tests\Feature\ProfileTest
✓ profile page is displayed
✓ profile information can be updated
✓ email verification status is unchanged when email is unchanged
✓ user can delete their account
✓ correct password must be provided to delete account

PASS Tests\Feature\TrainerClassTest
✓ assigned trainer can view session
✓ other trainer cannot manage session
✓ assigned trainer can start session
✓ trainer can mark attendance
✓ completing session marks pending members as no show

Tests: 132 passed (360 assertions)
Duration: 68.42s
```

php artisan test

El proyecto utiliza PHPUnit junto con las herramientas de pruebas proporcionadas por Laravel. Las pruebas funcionales se encuentran principalmente dentro de tests/Feature.

CONCLUSIONES

El desarrollo del Sistema de Gestión de Gimnasio permitió construir una aplicación web que centraliza diferentes procesos administrativos y operativos necesarios para el funcionamiento de un gimnasio. La implementación de módulos independientes facilita la administración de usuarios, membresías, pagos, clases grupales, asistencia, entrenamientos, nutrición y reportes.

La utilización del framework Laravel permitió organizar el sistema utilizando rutas, controladores, modelos, middleware, servicios, repositorios y vistas, logrando una estructura que facilita la separación de responsabilidades y el mantenimiento del código.

El uso de migraciones permite controlar la estructura de la base de datos desde el código fuente, haciendo posible que los diferentes integrantes del equipo puedan mantener una estructura de datos consistente durante el desarrollo.

La implementación de autenticación y control de acceso mediante roles permite limitar las acciones disponibles según el tipo de usuario. De esta forma, administradores, recepcionistas, entrenadores y socios disponen únicamente de las funciones correspondientes a sus responsabilidades dentro del sistema.